

EXPERIMENT - 3

Fundamentals of Errors and Debugging

Theory:

Developing a program is a step-by-step process, and it is common for programmers to make mistakes while writing code. These mistakes are called errors and can prevent the program from working as expected. Some errors stop the program from compiling, while others allow it to run but produce incorrect results. Understanding the different types of errors is important because it helps programmers identify problems more quickly and improve the quality of their programs.

The process of locating, analyzing, and correcting these mistakes is known as debugging. Debugging is an essential part of software development because it ensures that a program works correctly, produces accurate results, and performs reliably.

Different Types of Errors in C

Errors in C programming are generally divided into five major categories based on when and how they occur.

1. Syntax Errors

Syntax errors occur when the programmer does not follow the grammatical rules of the C language. Since the compiler checks the syntax before creating the executable file, these errors are detected immediately during compilation.

Some common reasons for syntax errors are:

- Forgetting to place a semicolon (;)
- Missing or extra braces ({ })
- Misspelling reserved keywords

- Incorrect use of parentheses or quotation marks

Result: The program cannot be compiled until all syntax errors are corrected.

2. Compilation (Type) Errors

Compilation errors, also known as type errors, occur when the program contains incorrect declarations or incompatible data types. Although the syntax may be correct, the compiler cannot process the program because of these issues.

Examples include:

- Using a variable before declaring it
- Assigning a floating-point value to an incompatible data type
- Calling a function with incorrect parameters

Result: The compiler reports the error, and the program must be fixed before moving to the next stage.

3. Linker Errors

After the source code has been compiled successfully, the linker combines different program files and libraries to create an executable file. If any required function or library is missing, a linker error occurs.

Typical causes include:

- Declaring a function but not defining it
- Forgetting to include the required library
- Missing the main() function Example: undefined reference to

'main'

Result: The executable file cannot be generated until all missing references are resolved.

4. Runtime Errors

Runtime errors appear only when the program is executed. These errors cannot be detected by the compiler because they depend on the program's execution and input values.

Some common runtime errors are:

- Division by zero
- Accessing memory that does not belong to the program
- Array index going beyond its valid limit
- Null pointer access

Result: The program may terminate unexpectedly, freeze, or produce unpredictable behaviour.

5. Logical Errors

Logical errors are considered the most difficult to identify because the program compiles and runs without showing any error messages. However, the output produced is not what the programmer intended.

These errors usually occur due to:

- Incorrect formulas or calculations
- Wrong conditions in if statements
- Improper looping logic
- Mistakes in the program's algorithm

Logical errors are usually detected by comparing the actual output with the expected output and carefully debugging the program.

Importance of Debugging

Debugging plays a significant role in software development. It allows programmers to:

- Identify the exact location of an error.

- Understand why the error occurred.
- Correct the mistake without affecting other parts of the program.
- Improve the overall quality and reliability of the software.

Without proper debugging, even a small mistake can lead to incorrect results or unexpected program failures.

Why Does a Program Produce Incorrect Output Even After Successful Compilation?

A successful compilation only confirms that the program follows the syntax rules of the C language and that the data types are used correctly. The compiler does not examine whether the programmer's logic is correct. Therefore, if the algorithm or calculations contain mistakes, the program will execute normally but generate incorrect results. Such problems are known as logical errors, and they can only be discovered through testing, verification, and debugging.

Incorrect Code 1 Input:

```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a= 10
    printf("%d\n", a);
    return 0;
}
```

Output:

```
(kali@kali)-[~]
$ gcc code1.c -o code1
code1.c: In function 'main':
code1.c:5:9: error: expected ',', or ';' before 'printf'
   5 |         printf("%d\n", a);
     |         ^~~~~~
```

Error Message:

error: expected ',', or ';' before 'printf'

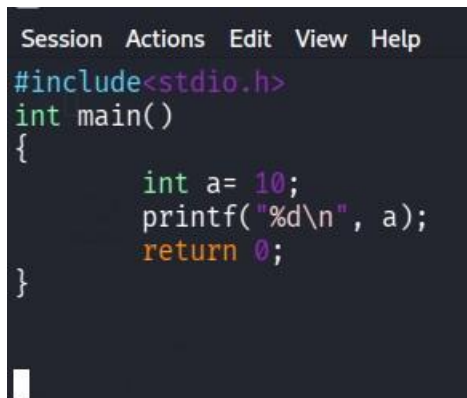
Error Type:

Syntax Error

Explanation:

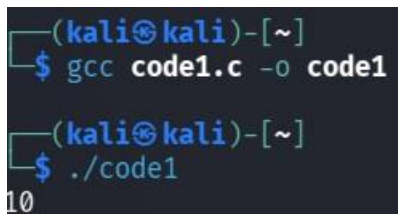
The semicolon (;) is missing after `int a = 10`. Because of this, the compiler treats the next line as part of the previous statement and reports a syntax error.

Correct Code:



```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a= 10;
    printf("%d\n", a);
    return 0;
}
```

Correct Output:



```
(kali㉿kali)-[~]
$ gcc code1.c -o code1

(kali㉿kali)-[~]
$ ./code1
10
```

Conclusion:

Adding the missing semicolon fixes the syntax error and allows the program to compile successfully.

Compiler Warnings (-Wall, -Wextra)

Theory

Compiler warnings are messages generated by the compiler to indicate possible problems in the program. Although warnings do not stop the program from compiling, they should never be ignored because they may lead to unexpected behaviour later. Using compiler options like `-Wall` and `-Wextra` helps

programmers detect issues such as unused variables, suspicious code, and possible logical mistakes. Fixing compiler warnings makes the program more reliable and easier to maintain.

- -Wall: Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall. **incorrect**

Code 2:

```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a;
    printf("hello\n");
    return 0;
}
```

Output:

```
(kali㉿kali)-[~]
$ gcc code2.c -o code2

(kali㉿kali)-[~]
$ ./code2
hello

(kali㉿kali)-[~]
$ gcc -Wall -Wextra code2.c
code2.c: In function 'main':
code2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
   4 |         int a;
     |             ^
```

Error Type:

Unused variable warning

Error Message / Output:

warning: unused variable 'a'

Explanation:

Variable 'a' is declared but never used. Remove it or use it.

Correct Code:

```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    printf("hello\n");
    return 0;
}
```

Correct Output:

```
(kali@kali)-[~]
$ gcc -Wall -Wextra code2.c

(kali@kali)-[~]
$ ./code2
hello
```

Conclusion:

After correcting the code, the program works as expected.

printf()-Based Debugging

Theory:

One of the simplest methods of debugging is using printf() statements. By printing the values of variables and checking the program flow, a programmer can understand how the program is executing. This method is very useful for finding logical errors because it allows the programmer to observe changes in variables during each step of execution. Before using advanced debugging tools, many programmers first use printf() statements to identify the source of a problem.

Incorrect Code 3:

Input:

```
Session Actions Edit View Help
#include<stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i=0; i<5; i++)
    {
        printf("DEBUG: entering iteration i=%d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("final sum= %d\n", sum);
    return 0;
}
```

Output:

```
(kali㉿kali)-[~]
└─$ gcc code3.c -o code3

(kali㉿kali)-[~]
└─$ ./code3
DEBUG: entering iteration i=1
DEBUG: sum updated to = 1
DEBUG: entering iteration i=2
DEBUG: sum updated to = 3
DEBUG: entering iteration i=3
DEBUG: sum updated to = 6
DEBUG: entering iteration i=4
DEBUG: sum updated to = 10
final sum= 10
```

Error Type:

Loop logic bug

Error Message / Output:

Program gives incorrect sum.

Correct Code:

```
Session Actions Edit View Help
#include<stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i=1; i<=5; i++)
    {
        printf("DEBUG: entering iteration i=%d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("final sum= %d\n", sum);
    return 0;
}
```

Correct Output:

Final Sum = 15

```
(kali㉿kali)-[~]
$ gcc -Wall -Wextra code3.c -o code3

(kali㉿kali)-[~]
$ ./code3
DEBUG: entering iteration i=1
DEBUG: sum updated to = 1
DEBUG: entering iteration i=2
DEBUG: sum updated to = 3
DEBUG: entering iteration i=3
DEBUG: sum updated to = 6
DEBUG: entering iteration i=4
DEBUG: sum updated to = 10
DEBUG: entering iteration i=5
DEBUG: sum updated to = 15
final sum= 15
```

Conclusion:

After correcting the code, the program works as expected.

Introduction to GDB (GNU Debugger)

Theory:

GDB (GNU Debugger) is a powerful debugging tool used for C programs. It allows programmers to execute a program step by step, inspect variables, set

breakpoints, and identify the exact location where an error occurs. GDB is especially useful for finding logical errors, runtime errors, and crashes. It helps programmers understand how the program behaves during execution and makes debugging more efficient.

Installation and compilation for GDB

Update package list and install GDB on Linux systems

\$ sudo apt update

\$ sudo apt install gdb

Compile source code with debugging symbols enabled (-g)

\$ cc -g -Wall -Wextra program.c -o program

Launch GDB with the compiled binary

\$ gdb ./program

Common GDB Commands

- **break (b):** Used to set a breakpoint at a specific line or function. The program pauses at that point so you can inspect it.
- **run (r):** Starts the execution of the program from the beginning.
- **next (n):** Executes the next line of code without entering a function call.
- **step (s):** Executes the next line of code and enters the function if one is called.
- **print (p):** Displays the current value of a variable or expression.
- **display:** Automatically shows the value of a variable after each debugging step.
- **continue (c):** Continues the execution of the program until the next breakpoint or until the program finishes.
- **list (l):** Displays the source code around the current line being executed.
- **info locals:** Shows the values of all local variables in the current function.
- **backtrace (bt):** Displays the sequence of function calls that led to the current point or error.
- **info breakpoints:** Lists all the breakpoints that have been set in the program.

- delete: Removes one or more breakpoints.
- disable: Temporarily disables a breakpoint without deleting it.
- enable: Re-enables a previously disabled breakpoint.
- quit (q): Exits the GDB debugger.

Incorrect Code 4:

```

Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int i, sum= 0;
    for(i=1; i<5; i++)
    {
        sum=sum+i;
    }
    printf("sum=%d\n", sum);
    return 0;
}

```

Output:

```

(kali@kali)-[~]
$ gcc -Wall -Wextra code4.c -o code4

(kali@kali)-[~]
$ ./code4
sum=10

```

Error Type:

Wrong loop condition

Error Message / Output: Actual

output: 10 instead of 15

Explanation:

Condition should be $i \leq 5$ instead of $i < 5$.

Correct Code:

```
Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int i, sum= 0;
    for(i=1; i<=5; i++)
    {
        sum=sum+i;
    }
    printf("sum=%d\n", sum);
    return 0;
}
```

Correct Output:

Sum = 15

```
(kali@kali)-[~]
$ gcc -Wall -Wextra code4.c -o code4

(kali@kali)-[~]
$ ./code4
sum=15
```

Conclusion:

After correcting the code, the program works as expected.

Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Incorrect code 5:

```

Session Actions Edit View Help
#include<stdio.h>
int square(int x)
{
    int result1;
    result1= x*x;
    return result1;
}
int main()
{
    int result, n=5;
    result = square(n);
    printf("square=%d\n", result);
    return 0;
}

```

Output:

```

(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE5.C -o SAMPLE5

(kali㉿kali)-[~]
$ ./SAMPLE5
Square = 10

```

Error Type:

Debugger demonstration **Error**

Message / Output:

No code error; demonstrates stepping.

Explanation:

Use 'next' to step over and 'step' to step into functions.

Correct Code:

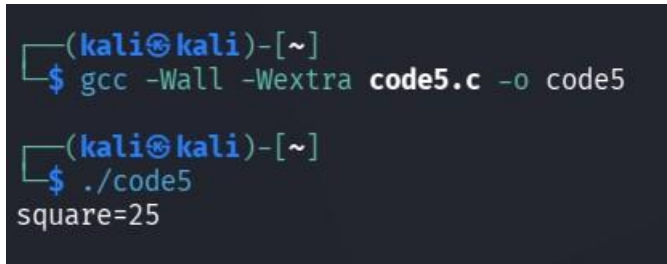
```

Session Actions Edit View Help
#include<stdio.h>
int square(int x)
{
    int result1;
    result1= x*x;
    return result1;
}
int main()
{
    int result, n=5;
    result = square(n);
    printf("square=%d\n", result);
    return 0;
}

```

Correct Output:

Square = 25

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is gcc -Wall -Wextra code5.c -o code5. The second command is ./code5. The output is square=25.

```
(kali@kali)-[~]  
$ gcc -Wall -Wextra code5.c -o code5  
  
(kali@kali)-[~]  
$ ./code5  
square=25
```

Conclusion:

After correcting the code, the program works as expected.

Understanding Stack Frames

- A stack frame contains information about a function that is currently being executed.
- When one function calls another, a new stack frame is created.
- Frame #0 represents the function where the error or crash actually occurred.
- Frame #1 represents the function that called Frame #0.
- Frame #2 represents the main function or the function that started the program.
- Stack frames help programmers trace the execution flow and identify the exact location of an error.

Incorrect Code 6:

```

Session Actions Edit View Help
#include<stdio.h>
int divide(int a, int b)
{
    return a/b;           //division by zero when b=0
}
int calculate(int x)
{
    return dicide(x,0);  // paaes 0 as divisor
}
int main()
{
    int result= calculate(10);
    printf("result: %d\n", result);
    return 0;
}
~

```

Output:

```

(kali@kali)-[~]
$ gcc -Wall -Wextra code6.c -o code6

(kali@kali)-[~]
$ ./code6
zsh: floating point exception ./code6

(kali@kali)-[~]

```

Error Type:

Division by zero

Error Message / Output:

SIGFPE / Arithmetic exception

Explanation:

Dividing by zero causes a runtime crash.

Correct Code:

```

Session Actions Edit View Help
#include<stdio.h>
int divide(int a, int b)
{
    if(b==0)
    {
        printf("division by zero is not allowed.\n");
        return 0;
    }
    return a/b;
}

int calculate(int x)
{
    return divide(x,2);
}

int main()
{
    int result= calculate(10);
    printf("result:%d\n", result);
    return 0;
}

```

Correct Output:

Result after valid divisor

```

(kali@kali)-[~]
$ gcc -Wall -Wextra code6.c -o code6

(kali@kali)-[~]
$ ./code6
result:5

```

Conclusion:

After correcting the code, the program works as expected.

Debugging Array Out-of-Bounds Errors using GDB

An array out-of-bounds error occurs when a program tries to access an array element outside its valid range. In C, the compiler does not automatically check array boundaries. Accessing an invalid index can cause undefined behaviour, memory corruption, or even a program crash. GDB helps programmers inspect array addresses and identify invalid memory access. To avoid this error, always use the correct loop condition and ensure that array indices remain within the valid range.

Incorrect code 7:

```

Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a[5];
    int i;
    for(i=0; i<=5; i++)
    {
        scanf("%d",&a[i]);
    }
    return 0;
}

```

Output:

```

(kali㉿kali)-[~]
$ vi code7.c

(kali㉿kali)-[~]
$ gcc -Wall -Wextra code7.c -o code7

(kali㉿kali)-[~]
$ ./code7

```

Error Type: Array index

error Error Message:

Array Out-of-Bounds Error

Explanation:

Valid indices are 0 to 4. It will not give us output **Correct**

Code:

```

Session Actions Edit View Help
#include<stdio.h>
int main()
{
    int a[5];
    int i;
    for(i=0; i<5; i++)
    {
        scanf("%d",&a[i]);
    }
    return 0;
}

```

Correct Output:

Runs safely

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra code7.c -o code7  
  
(kali㉿kali)-[~]  
$ ./code7
```

Conclusion:

After correcting the code, the program works as expected.

Segmentation Faults (SIGSEGV) using GDB

- A Segmentation Fault (SIGSEGV) occurs when a program tries to access memory that it is not allowed to access. This usually happens because of a NULL pointer, an uninitialized pointer, or invalid memory access. GDB helps locate the exact line where the segmentation fault occurs. By checking pointer values in GDB, programmers can identify the cause of the crash and correct the program. Proper pointer initialization and valid memory access help prevent segmentation faults.

Incorrect code 8:

```
#include<stdio.h>  
int main()  
{  
    int*p= NULL;  
    *p=10;  
    printf("%d\n",*p);  
    return 0;  
}
```

Output:

```
(kali㉿kali)-[~]  
$ gcc code8.c -o code8  
  
(kali㉿kali)-[~]  
$ ./code8  
zsh: segmentation fault ./code8
```

Error Type:

NULL pointer dereference **Error**

Message / Output:

Segmentation fault **Explanation:**

Writing through a NULL pointer causes segmentation fault.

Correct Code:

```
#include<stdio.h>
int main()
{
    int x;
    int *p=&x;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

Correct Output:

10

```
(kali㉿kali)-[~]
$ gcc code8.c -o code8

(kali㉿kali)-[~]
$ ./code8
10
```

Conclusion:

After fixing the identified errors, the program executes correctly and produces the expected output. Debugging improves the accuracy and reliability of a program. Testing the program after every correction helps ensure that no new errors are introduced.

Systematic Debugging Strategy:

1. A systematic approach makes debugging more efficient. The following steps are commonly followed:
2. Compare the actual output with the expected output.

3. Reproduce the error to understand when it occurs.
4. Compile the program using options such as -Wall, -Wextra, and -g to detect warnings and enable debugging.
5. Locate the section of code responsible for the error.
6. Use printf() statements or GDB to examine variable values and program execution.
7. Correct only the required part of the code.
8. Recompile and run the program to verify the fix.
9. Finally, test the program with different input values to ensure that the error has been completely resolved and that the program works correctly under various conditions.

GNU Concept

What is the GNU Project?

The GNU Project is a free and open-source software project started by Richard Stallman in 1983. Its main aim is to create a Unix-like operating system that anyone can use, modify, and share freely. The GNU Project provides many useful software development tools such as the GCC compiler, GDB debugger, libraries, and other utilities that are widely used by programmers.

How Does GDB Fit into the GNU Project?

GDB (GNU Debugger) is one of the important tools developed under the GNU Project. It helps programmers find and fix errors in their programs by allowing them to:

- Run the program step by step.
- Set breakpoints to pause execution.
- Check the values of variables and memory.
- Trace function calls.
- Find the cause of runtime errors and program crashes.

GDB makes debugging easier and helps developers understand how a program behaves during execution.

Shell Context

Shell:

A Shell is a program that acts as an interface between the user and the operating system. It accepts commands from the user, sends them to the kernel, and displays the output. Most Linux users interact with the system through a shell.

Kernel:

The Kernel is the core part of an operating system. It manages the computer's hardware resources such as the CPU, memory, storage, and input/output devices. It also acts as a bridge between the hardware and the software.

Common Linux Shells::

Bash:

The default shell used in most Linux distributions. It is simple, reliable, and widely used.

Zsh:

A more advanced shell that provides better customization, auto-completion, and plugins.

Fish:

A user-friendly shell that offers syntax highlighting, command suggestions, and an easy-to-use interface.

Difference Between Breakpoint, Watchpoint, Catchpoint, and Tracepoint

1.Breakpoint

A breakpoint is used to pause the execution of a program at a specific line of code or function. It allows the programmer to inspect variables, memory, and

the flow of the program before continuing execution. Breakpoints are mainly used to check whether a particular part of the program is working correctly.

2. Watchpoint

A watchpoint monitors the value of a variable or memory location. Unlike a breakpoint, it does not stop at a specific line of code. Instead, it pauses the program whenever the selected variable changes its value. This is useful for finding unexpected changes in variables during execution.

3. Catchpoint

A catchpoint stops the program when a particular event occurs, such as an exception, a signal, or the loading of a shared library. It is mainly used to detect special events that are not directly related to a specific line of code.

4. Tracepoint

A tracepoint records information about the program while it is running without stopping its execution. It is useful for monitoring the behaviour of a program, especially large or long-running applications, where stopping the program repeatedly is not practical.

Summary:

- A breakpoint pauses the program when execution reaches a specified line or function.
- A watchpoint pauses the program when the value of a selected variable or memory location changes.
- A catchpoint pauses the program when a predefined event, such as an exception or signal, occurs.
- A tracepoint records debugging information without interrupting the execution of the program.

